

Quick Sort

It is also called the partition exchange sort. It is an efficient algorithm and it passes a very good time complexity in average case.

Quick Sort (A, p, r)

If $p < r$ then

$q \leftarrow \text{partition}(A, p, r)$

Quick Sort ($A, p, q-1$)

Quick Sort ($A, q+1, r$)

Partitioning of array

$\text{partition}(A, p, r)$

$x \leftarrow A[r]$

$i \leftarrow p - 1$

for $j \leftarrow p$ to $r-1$

do if $A[j] \leq x$

then $i \leftarrow i+1$

exchange $A[i] \leftrightarrow A[j]$

exchange $A[i+1] \leftrightarrow A[r]$

return $i+1$

0	1	2	3	4	5	6	7	8	9	10
36	15	40	1	60	20	55	25	50	20	

1

$p=1$ $r=10$

$x = A[10] = 20$

$i = p-1 = 1-1 = 0$

$j = 1$ to 9

Now $j=1$ and $i=0$

$$A[1] = 36$$

$$36 \leq 20 \text{ [False]}$$

$j=2$ and $i=0$

$$A[2] = 15$$

$$15 \leq 20 \text{ [True]}$$

$$i = 0 + 1 = 1$$

$$A[1] \leftrightarrow A[2]$$

1	2	3	4	5	6	7	8	9
15	36	40	1	60	20	55	25	50

Now $j=3$ and $i=1$

$$A[3] = 40$$

$$40 \leq 20 \text{ [False]}$$

$j=4$ and $i=1$

$$A[4] = 1$$

$$1 \leq 20 \text{ [True]}$$

$$i = 1 + 1 = 2$$

$$A[2] \leftrightarrow A[4]$$

1	2	3	4	5	6	7	8	9	10
15	1	40	36	60	20	55	25	50	20

Now $j=5$ and $i=2$

$$A[5] = 60$$

$$60 \leq 20 \text{ [False]}$$

$j=6$ and $i=2$

$$A[6] = 20$$

$$20 \leq 20 \text{ [True]}$$

$$i = 2 + 1 = 3$$

$$A[3] \leftrightarrow A[6]$$

1	2	3	4	5	6	7	8	9	10
15	1	20	36	60	40	55	25	50	20

Now $j=7, i=3$
 $A[7]=55$

$55 < 20$ [False]

$j=8, i=3$
 $A[8]=25$

$25 < 20$ [False]

$j=9, i=3$
 $A[9]=50$

$50 < 20$ [False]

Now $A[i+1]$ i.e. $A[4] \leftrightarrow A[10]$

1	2	3	4	5	6	7	8	9	
15	1	20	20	60	40	55	25	50	36

1	2	3	4	5	6	7	8	9	10
15	1	20	20	60	40	55	25	50	36

15	1	20	20	60	40	55	25	50	36
----	---	----	----	----	----	----	----	----	----

- Quick Sort is stable sort
- Worst case time complexity is $O(n^2)$
- Average case time complexity is $O(n \log n)$
- Best case time complexity is $O(n \log n)$

Bubble Sort

In Bubble Sort, each element is compared with its adjacent element. If the first element is larger than the second one, then the positions of the elements are interchanged, otherwise it is not changed.

Algorithm

Let A be a linear array of n numbers. Swap is a temporary variable for swapping the positions of the numbers.

Input n number of an array.

Initialize $i=0$ and repeat through step 4 if $(i < n)$

Initialize $j=0$ and repeat through step 4 if $(j < n-i-1)$

if $[A[j] > A[j+1]]$

Swap = $A[j]$

$A[j] = A[j+1]$

$A[j+1] = \text{Swap}$

Display the sorted no. of list.

Exit

42 33 23 74 44

$n=5$

$i=0$, $j=0$ to $(5-0-1)=4$

First pass

0	42	] swap	33		33		33		33
1	33		42	] swap	23		23		23
2	23		23		42	] No swapping	42		42
3	74		74		74		74		74
4	44		44		44		44	] swap	44
									74

Second pass

$i=1$

$j = 0 \text{ to } 2 (> 5-1-1)$

0	33	23	23
1	23	33	33
2	42	42	42
3	44	44	44
4	74	74	74

Annotations:
 - Between column 1 and 2: swap for rows 0 and 1.
 - Between column 2 and 3: No swap for rows 1, 2, 3, and 4.

Third pass

$i=2$

$j = 0 \text{ to } 1 (> 5-2-1)$

0	23
1	33
2	42
3	44
4	74

Annotation:
 - Between column 0 and 1: No swap for row 0.

Best case complexity - $O(n)$

Worst case complexity - $O(n^2)$

Average case complexity - $O(n^2)$

Heap Sort

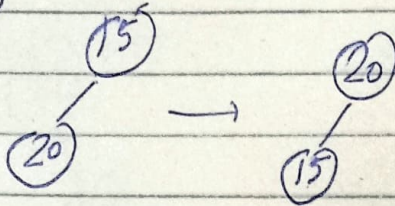
1	2	3	4	5
15	20	7	9	30

- ① To create a max heap
- ② Delete the element from one by one at root.

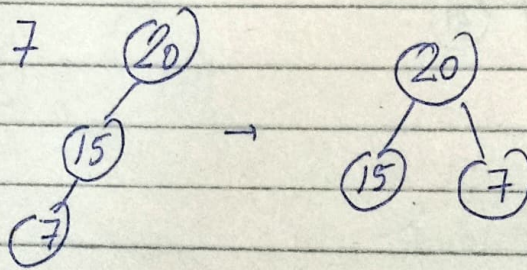
First element is 15

(15)

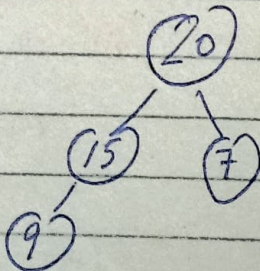
Next element 20

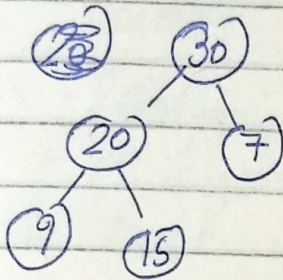
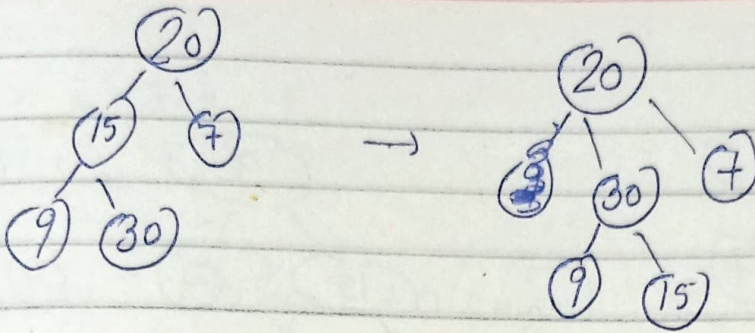


Next element 7



Next element 9

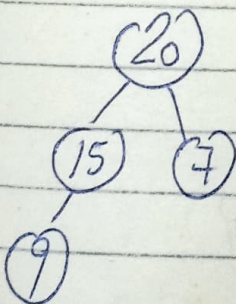
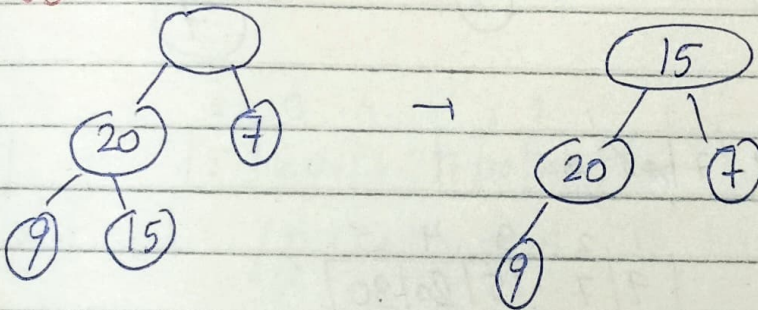




A

1	2	3	4	5
30	20	7	9	15

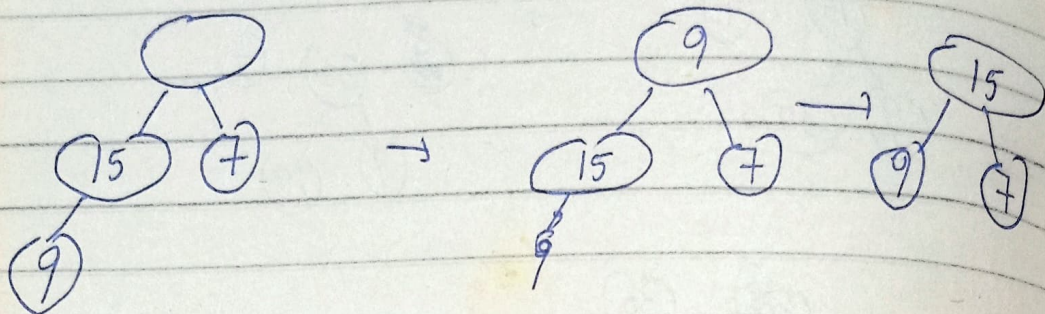
Delete 30



1	2	3	4	5
15	20	7	9	30

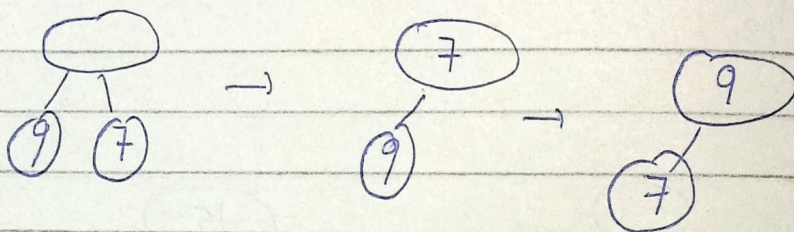
1	2	3	4	5
20	15	7	9	30

Delete 20



1	2	3	4	5
25	9	7	20	30

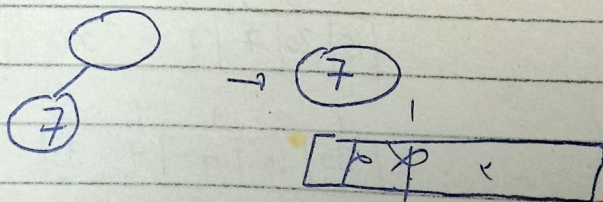
Delete 15



1	2	3	4	5
9	7	15	20	30

1	2	3	4	5
9	7	15	20	30

Delete 9



1	2	3	4	5
7	9	15	20	30

Delete 7.

1	2	3	4	5
7	9	15	20	30

Time complexity = $O(n \log n)$ + $O(n \log n)$
 ↓ ↓
 Insertion Deletion

$$= \cancel{20n \log n} \quad O(2n \log n)$$

$$= O(n \log n)$$

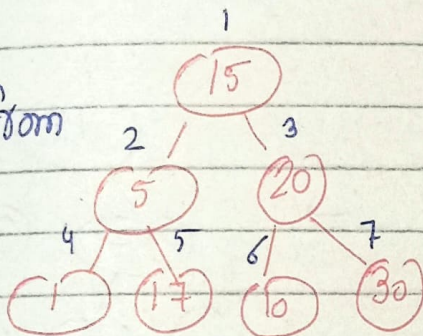
- Unstable Box E

- Inplace Sort.

Heapify

It apply from

non-leaf
node

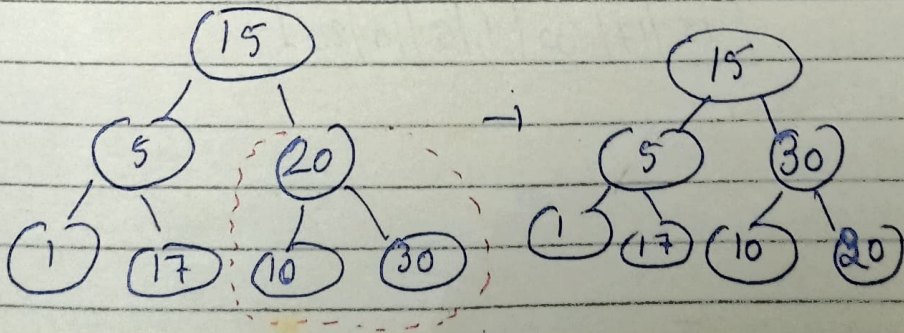


ACI

	1	2	3	4	5	6	7
	15	5	20	1	17	10	30

leaf node = $\lfloor \frac{n}{2} \rfloor + 1$ to n

$$= \left\lceil \frac{7}{2} \right\rceil + 1 = 4 \text{ to } 7$$



1	2	3	4	5	6	7
15	5	20	1	17	10	30

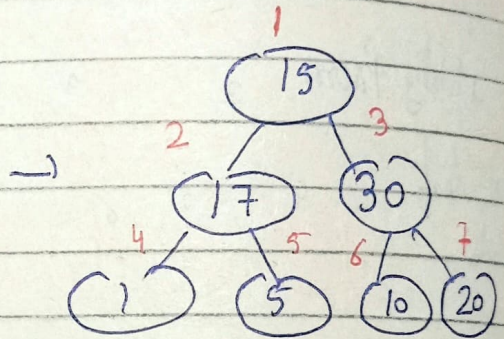
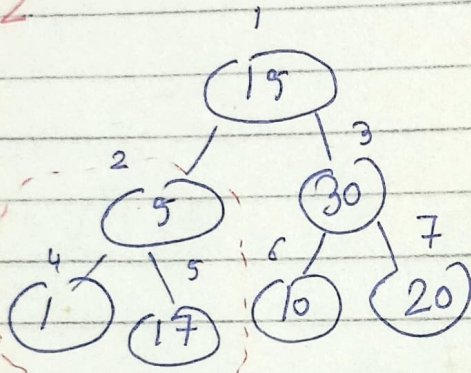
$$i = \left\lfloor \frac{n}{2} \right\rfloor = \left\lfloor \frac{87}{2} \right\rfloor = 3$$

$$\text{Left child} = 2 \times i = 2 \times 3 = 6$$

$$\text{Right child} = 2 \times i + 1 = 2 \times 3 + 1 = 7$$

1	2	3	4	5	6	7
15	5	30	1	17	10	20

$$i = 2$$



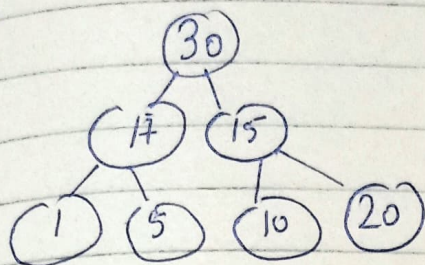
1	2	3	4	5	6	7
15	5	30	1	17	10	20

$$\text{Left child} = 2 \times i = 2 \times 2 = 4$$

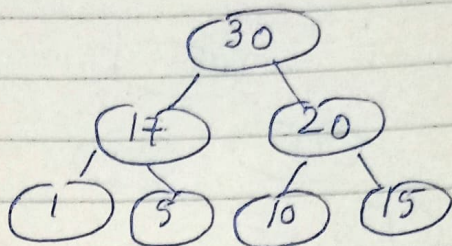
$$\text{Right child} = 2 \times i + 1 = 2 \times 2 + 1 = 5$$

1	2	3	4	5	6	7
15	17	30	1	5	10	20

$i=1$



→



1	2	3	4	5	6
15	17	30	15	10	20

1	2	3	4	5	6	7
15	17	30	1	15	10	20

Left child = $2 \times i = 2 \times 1 = 2$

Right child = $2 \times i + 1 = 2 \times 1 + 1 = 3$

1	2	3	4	5	6	7
30	17	15	1	5	10	20

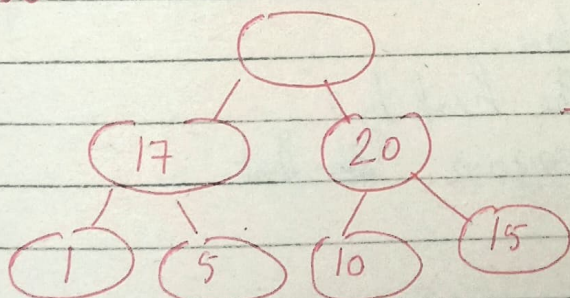
$i=3$

Left child = $2 \times i = 2 \times 3 = 6$

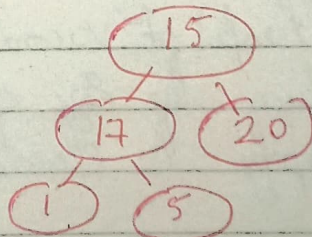
Right child = $2 \times i + 1 = 2 \times 3 + 1 = 7$

30	17	20	1	5	10	15
----	----	----	---	---	----	----

Delete 30



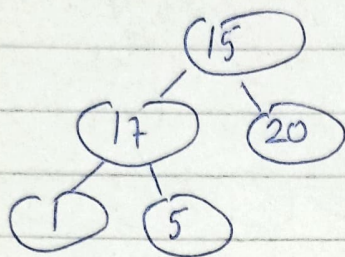
→



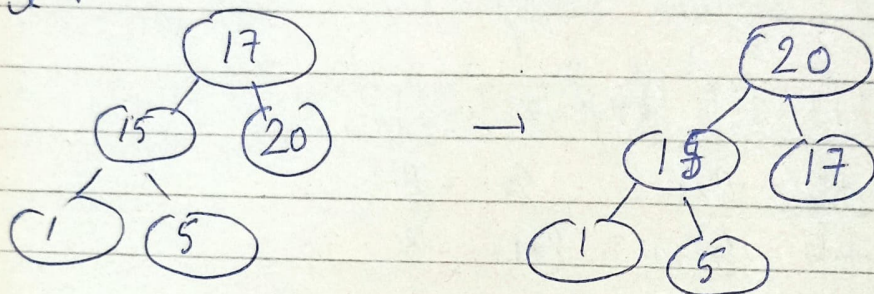
call max-heapify

$i = \left\lfloor \frac{n}{2} \right\rfloor = \left\lfloor \frac{6}{2} \right\rfloor = 3$

$i=2$



$i=1$



1	2	3	4	5	6
20	15	17	1	5	30

Working of heap Sort

- (I) Build a heap (Max or Min heap)
- (II) Remove the root element and put ~~the~~ at the end of the array (n^{th} position). Put the last item of the tree (heap) at vacant place.
- (III) Reduce the size of the heap by 1.
- (IV) Heapify the element again. ~~so that~~